SiR^2ENA

Simple Register-Register Extensible Neo-Architecture

CPU Developer Reference

1.0

October 25, 2026

Contents

1 Preface **3**

2 Glossary **3**

3 Introduction **3**

3.1 Architectural details 3

4 Registers **4**

4.1 Pointer registers 5

4.1.1 (0x1D) rfp - frame pointer 5

4.1.2 (0x1E) rrap - interrupt routine array pointer 5

4.1.3 (0x13) rlfp - last frame pointer 5

4.1.4 (0x10) rip - instruction pointer 5

4.1.5 (0x11) rtp - procedure tail pointer 5

4.1.6 (0x12) rsp - stack pointer 5

4.1.7 (0x14) rpdp - page directory pointer 5

4.2 Restricted purpose 5

4.2.1 (0x1F) rz - zero register 5

4.2.2 (0x16) rpl - current privilege level 6

4.3 General purpose 6

4.3.1 (0x00) r0 6

4.3.2 (0x01) r1 6

4.3.3 (0x02) r2 6

4.3.4 (0x03) r3 6

4.3.5 (0x04) r4 6

4.3.6 (0x05) r5 6

4.3.7 (0x06) r6 6

4.3.8 (0x07) r7 6

4.3.9 (0x08) r8 6

4.3.10 (0x09) r9 6

4.3.11 (0x0A) r10 6

4.3.12 (0x0B) r11 7

4.3.13 (0x0C) r12 7

4.3.14 (0x0D) r13 7

4.3.15 (0x0E) r14 . . . 7
4.3.16 (0x0F) r15 . . . 7
4.4 Special registers . . . 7
4.4.1 (0x15) rcf . . . 7
4.4.2 (0x18) rfi0 . . . 7
4.4.3 (0x19) rfi1 . . . 7
4.4.4 (0x1A) rfo0 . . . 7
4.4.5 (0x1B) rfo1 . . . 8

5 Memory access **8**

6 Boot-up process **8**
6.1 Boot-up Late Initialisation Program . . . 8

7 Operation cycle **9**

8 Arithmetic Logic Unit **9**
8.1 Operations . . . 9

9 Interrupts **9**
9.1 Interrupt List . . . 10
9.1.1 #UDO Undefined Opcode . . . 10
9.1.2 #IOP Invalid Operation . . . 10
9.1.3 #PLE Page Look-up Error . . . 11
9.1.4 #MAE Memory Access Exception . . . 11
9.1.5 #GPE General Protection Error . . . 11
9.2 Interrupt Routine Array . . . 12
9.2.1 Interrupt Routine Array Entry . . . 12
9.3 Interrupt Routine Handler . . . 12
9.4 User-defined interrupts . . . 12

10 Memory management **12**
10.1 Paging . . . 12
10.1.1 Page Directory . . . 13
10.1.2 Page Directory Entry . . . 13
10.1.3 Page Table . . . 13
10.1.4 Page Descriptor . . . 13
10.1.5 Setting up paging . . . 14
10.2 Virtual memory . . . 14
10.3 Access protection . . . 14
10.4 Level-0 physical memory map . . . 14
10.5 Level-1 physical memory map . . . 14
10.6 Memory Management Unit . . . 15
10.6.1 Operations . . . 15

11 Protection domains **16**
11.1 Protection level transfer . . . 16

12 Context switching **17**

13 Hardware features **17**
13.1 Feature ID table 17
13.2 Programmable Interval Timer 17
13.2.1 0x00 Set mode 17
13.2.2 0x01 Set tick rate 18
13.2.3 0x02 Set target INT 18
13.2.4 0x03 Get tick count 18
13.2.5 0x04 Reset counter 18
13.2.6 Feature information 18
13.3 UART 18
13.3.1 TX state 19
13.3.2 RX state 19
13.3.3 Configuration packet 19
13.3.4 Interrupts 19
13.4 SPI 19

14 Structures **20**
14.1 CPUID 20

1 Preface

2 Glossary

Acronyms

BLIP Boot-up Late Initialisation Program. 8, 12, 13

CPU Central Processing Unit. 3

CU Control Unit. 8, 14

MMIO Memory-Mapped Input/Output. 3, 13

3 Introduction

This document complements the `SiRENA` computer architecture by specifications for the Central Processing Unit (CPU). It is to be used as a reference for both software and hardware developers for functions and working details of the CPU.

3.1 Architectural details

- *Little-endian* memory model
- *Paging* for memory separation
- *MMIO* for device I/O

- *2-level* protection domain
- *Extensions* for optional features and additional instruction sets

4 Registers

The SiRENA architecture, similarily to the TMS9900, does not store general-purpose registers on the CPU, but rather a *frame pointer* that points to the address in *system RAM* of the current *register frame* for the executing context.

A *register frame* refers to a (virtual) structure in RAM of 128 bytes (1024 bits) in length that stores 32 4-byte values total:

Table 1: Register frame (1023-768)

1023..992	991..960	959..928	927..896	895..864	863..832	831..800	799..768
reserved				rfo1	rfo0	rfi1	rfi0

Table 2: Register frame (767-512)

767..736	735..704	703..672	671..640	639..608	607..576	575..544	543..512
reserved	rpl	rcf	rpdp	rlfp	rsp	rtp	rip

Table 3: Register frame (511-256)

511..480	479..448	447..416	415..384	383..352	351..320	319.288	287..256
r15	r14	r13	r12	r11	r10	r9	r8

Table 4: Register frame (255-0)

255..224	223..192	191..160	159..128	127..96	95..64	63..32	31..0
r7	r6	r5	r4	r3	r2	r1	r0

IMPORTANT: Every *register frame* used by the *target program* MUST be word-aligned and zeroed before first use.

IMPORTANT: Any used *register frame* MUST be located inside the physical memory region that maps *system RAM*. For more information see 10.4.

Registers are categorised into two categories:

- **Global** registers are stored directly on the CPU and their values persist between frame changes
- **Local** registers are stored in the *register frame* and their address and/or value depends on the current *frame pointer*

4.1 Pointer registers

4.1.1 (0x1D) rfp - frame pointer

Availability: Global
Write-restricted
Stores the physical address to the current *register frame*. It must point to an address within the *system RAM* (See 10.4); otherwise, the CPU raises a **#IOP** on write.

4.1.2 (0x1E) rrap - interrupt routine array pointer

Availability: Global
Write-restricted
Stores the physical address to the current Interrupt Routine Array. Set by the supervisor to indicate which IRA the CPU should use for interrupt handling

4.1.3 (0x13) rlfp - last frame pointer

Availability: Local
Read-only
Stores the physical address to the previous *register frame*. Used when returning from interrupt routines. **#IOP** is raised on a write attempt.

4.1.4 (0x10) rip - instruction pointer

Availability: Local
Stores the address of the currently executing instruction

4.1.5 (0x11) rtp - procedure tail pointer

Availability: Local
Stores the address of the instruction that called the current procedure

4.1.6 (0x12) rsp - stack pointer

Availability: Local
Stores the address of the current stack frame

4.1.7 (0x14) rpdp - page directory pointer

Availability: Local
Write-restricted
Stores the physical address of the level-0 page directory structure. If it points to 0, paging is disabled.

4.2 Restricted purpose

4.2.1 (0x1F) rz - zero register

Availability: Global
Read-only
Always 0. **#IOP** is raised on a write attempt.

4.2.2 (0x16) rpl - current privilege level

Availability: Local
Write-restricted
Stores the current privilege level

4.3 General purpose

4.3.1 (0x00) r0

Availability: Local

4.3.2 (0x01) r1

Availability: Local

4.3.3 (0x02) r2

Availability: Local

4.3.4 (0x03) r3

Availability: Local

4.3.5 (0x04) r4

Availability: Local

4.3.6 (0x05) r5

Availability: Local

4.3.7 (0x06) r6

Availability: Local

4.3.8 (0x07) r7

Availability: Local

4.3.9 (0x08) r8

Availability: Local

4.3.10 (0x09) r9

Availability: Local

4.3.11 (0x0A) r10

Availability: Local

4.3.12 (0x0B) r11

Availability: Local

4.3.13 (0x0C) r12

Availability: Local

4.3.14 (0x0D) r13

Availability: Local

4.3.15 (0x0E) r14

Availability: Local

4.3.16 (0x0F) r15

Availability: Local

4.4 Special registers

4.4.1 (0x15) rcf

Availability: Local

Stores the control flags for the current execution init and output flags of ALU/FPU operations

Table 5: Available control flags

Bit	Label	Name	Value
0	SF	Sign flag	1 if the MSB of the result is 1; 0 otherwise
1	ZF	Zero flag	1 if the result of an operation is zero; 0 otherwise
2	CF	Carry flag	1 if an addition operation produces a carry bit; 0 otherwise
3	OF	Overflow flag	1 if an overflow occurs during an operation; 0 otherwise

4.4.2 (0x18) rfi0

Availability: Local

Used for input flags for certain operations

4.4.3 (0x19) rfi1

Availability: Local

Used for input flags for certain operations

4.4.4 (0x1A) rfo0

Availability: Local

Used for output flags of certain operations

4.4.5 (0x1B) rfo1

Availability: Local
Used for output flags of certain operations

5 Memory access

All memory I/O requires the target address to be aligned to the boundary of the type that is being requested.

This means that word I/O requires the address to be aligned to `0x4`, half-word I/O requires the address to be aligned to `0x2`, and byte I/O is unrestricted (`0x1` alignment).

On incorrect address alignment the MMU raises a **#MAE**.

6 Boot-up process

At power-on/reset, an initial *register frame* is set up at the beginning of the *system RAM* with the following values:

Register	Value
r0-r15	0
rip	0xFFA0 0000
rtp	0x0
rsp	0x0
rpdp	0x0
rcf	0
rpl	1
rfi*n*	0
rfo*n*	0

As well as global registers having the following default values:

Register	Value
rfp	0xFFFF FF80
rrap	0x0

This means that the CPU starts executing from the *BLIP ROM*, which is where the Boot-up Late Initialisation Program (BLIP) is located. How this program is accessed by the CPU is up to the specific implementation; in most cases, it will live in a separate ROM chip that the MMU can address directly.

IMPORTANT: Any physical memory is NOT zeroed at start-up and as such must be treated as uninitialised, unless the loaded BLIP or target program explicitly does do.

6.1 Boot-up Late Initialisation Program

The function of the BLIP is final hardware and device initialisation before loading and handing over control to the *target program*. How the target program

is discovered and loaded is up to the specific implementation.

7 Operation cycle

The CPU operates at with standard fetch→decode→execute cycle. By the end of a single cycle, the CPU must advance the *instruction pointer* by a single word for any executed instruction that does not explicitly set the *instruction pointer*.

8 Arithmetic Logic Unit

The ALU is a component of the CPU responsible for integer arithmetic and logical operations.

8.1 Operations

The ALU must provide *at least* the following operations for the Control Unit (CU):

Mnemonic	Description	SF	ZF	CF	OF
iadd32c(a, b) → y	32-bit Add integer with carry	$y >> 31$	$y = 0$	carry	signed overflow
isub32(a, b) → y	32-bit Subtract integer	$y >> 31$	$y = 0$	borrow	signed overflow
imul32(a, b) → y	32-bit Multiply integer	$y >> 31$	$y = 0$	0	0
and32(a, b) → y	32-bit logical AND	$y >> 31$	$y = 0$	0	–
or32(a, b) → y	32-bit logical OR	$y >> 31$	$y = 0$	0	–
xor32(a, b) → y	32-bit logical XOR	$y >> 31$	$y = 0$	0	–
lshl32(a, n) → y	32-bit logical shift left	$y >> 31$	$y = 0$	last shifted bit	–
lshr32(a, n) → y	32-bit logical shift right	$y >> 31$	$y = 0$	last shifted bit	–
ashr32(a, n) → y	32-bit arithmetic shift right	$y >> 31$	$y = 0$	last shifted bit	–
signe18(a) → y	Sign-extend to 31 using bit 17 as sign	–	–	–	–
signe16(a) → y	Sign-extend to 31 using bit 15 as sign	–	–	–	–
signe8(a) → y	Sign-extend to 31 using bit 7 as sign	–	–	–	–
zeroe16(a) → y	Zero-extend from bit 16 to 31	–	–	–	–
zeroe8(a) → y	Zero-extend from bit 8 to 31	–	–	–	–

9 Interrupts

Interrupts are handled by the CPU after every execute cycle of a single instruction. The CPU checks if any interrupts were raised after executing the instruction and, if so, checks and picks the correct Interrupt Routine Handler from the current Interrupt Routine Array to handle the raised interrupt, switching to the *register frame* specified for the handler and filling in/out registers if necessary.

If the `enable` bit of the IRA entry is not set, the interrupt stops being handed over to be handled in software, but the exact behaviour depends on the type of the raised interrupt.

If the `rrap` register is set to 0, all interrupts except *faults*, which immediately prompt a CPU reset, are ignored.

There are three types of interrupts:

Type	Recoverable	Unhandled pauses execution
Signal	✓	✗
Trap	✓	✓
Fault	✗	✓

- *Signal* interrupts do not pause execution and do not have to be explicitly handled in any way. This type of interrupt is recommended for general hardware-software communication.
- Raised *trap* interrupts are most commonly recoverable errors and will pause execution of the frame that the trap occurred in if no valid IRH was found to handle the raised interrupt.
- Raised *fault* interrupts are non-recoverable and reset the CPU upon existing from the IRH or if no valid handler was found. These cannot be raised manually from software.

9.1 Interrupt List

ID	Mnemonic	Name	Type	Notes
0x00	–	Reserved	–	–
0x01	#UDO	Undefined Opcode	Trap	See 9.1.1
0x02	#DBG	Debug	Trap	
0x03	#BRK	Breakpoint	Trap	
0x04	#IOP	Invalid Operation	Trap	See 9.1.2
0x05	#DBZ	Division By Zero	Trap	
0x06	#PLE	Page Look-up Error	Trap	See 9.1.3
0x07	#MAE	Memory Access Exception	Fault	See 9.1.4
0x08	#GPE	General Protection Error	Trap	See 9.1.5
0x09 .. 0x0F				CPU reserved
0x10 .. 0x8F			Signal	User-defined
0x90 .. 0xFF			Signal	Hardware reserved

9.1.1 #UDO Undefined Opcode

rfi0		
31..9	8..4	3..0
Unused	Faulting instruction opcode	Faulting instruction opset

9.1.2 #IOP Invalid Operation

The *Invalid Operation* interrupt sets the `rfi0` register accordingly based on the type of the operation that caused the interrupt:

rfi0	Description
0x00	Write to read-only register
0x01	New frame pointer outside system RAM
0x02	Unknown feature requested
0x03	Out of range
0x04	Wrong I/O data size
0xFE	UART attempted write during transfer
0xFF	Other invalid operation

9.1.3 #PLE Page Look-up Error

Raised by the MMU when an error occurs during the look-up of a requested page.

rfi0
31..0
The virtual memory address that caused the error

rfi1	Description
0x00	Page not found

9.1.4 #MAE Memory Access Exception

Raised by the MMU when an error occurs during memory I/O.

rfi0
31..0
The physical memory address that caused the error

rfi1	Description
0x00	Unknown read error
0x01	Unknown write error
0x02	Incorrect address alignment
0x03	Invalid address
0x04	Write on read-only memory region

9.1.5 #GPE General Protection Error

Raised on any kind of unlawful operation.

rfi0	Description
0x00	Incorrect privilege level
0x01	Execute on no-execute page
0x02	Write on read-only page
0x03	Undefined privilege level

rfi1		
31..4	3..2	1..0
–	npl	cpl
Unused	Needed privilege level	Current privilege level

9.2 Interrupt Routine Array

Offset	Value
0x00	Entry for INT 0x00
0x0C	Entry for INT 0x01
0x18	Entry for INT 0x02
...	...
0xBF4	Entry for INT 0xFF
0xC00	

9.2.1 Interrupt Routine Array Entry

95..64	63..32	31..1	0
&rh	&rfp	–	en
Physical address of the IRH	Physical address of the register frame	Reserved	Enabled

NOTE: The *register frame* used by the IRH **must** be writeable and located in a R/W memory region.

9.3 Interrupt Routine Handler

Interrupt Routine Handlers (IRH) are *user-defined level-1* procedures that respond to a specific interrupt raised by the CPU. IRs have their own *register frame* and can access the previous *register frame* via its address located in the `rlfp` register.

IRHs can exit at any time and resume execution by calling `reti`. For non-recoverable interrupts this will result in the CPU being reset.

9.4 User-defined interrupts

Any ID in the *user-defined* range (See 9.1) can be used for custom interrupts. These can only be of the **signal** type.

10 Memory management

10.1 Paging

The SiRENA architecture utilises *paging* as its technique for memory protection and virtual address space mapping. A single page spans **4 KiB** (`0x1000`) of physical memory.

The page table is a 3-level nested structure; the full 32-bit virtual address space (0x0000 0000 - 0xFFFF FFFF) is encapsulated by a level-0 page directory which contains 256 entries, each pointing to a level-1 page directory. A level-1

page directory contains 256 entries, each pointing to a page table (or 0). Every page table entry (page descriptor) then describes the page as well as its physical address. This allows for mapping the entire address range without unnecessarily wasting too much memory for paging structures in situations that do not need the entire address range mapped.

10.1.1 Page Directory

The page directory is an linear array of 256 page directory entries

Offset	Value
0x00	Entry 0
0x04	Entry 1
0x08	Entry 2
...	...
0x3FC	Entry 255
0x400	

10.1.2 Page Directory Entry

The page directory entry specifies where in physical memory the targeted page directory or page table is located and its flags

31..4	3..1	0
&adr	–	p
Physical page directory/table address[31:4]	Reserved	Present

NOTE: `adr` and the target structure **must** be `0x10` aligned

10.1.3 Page Table

The page table is a linear array of 16 page descriptors

Offset	Value
0x00	Descriptor 0
0x04	Descriptor 1
0x08	Descriptor 2
...	...
0x3C	Descriptor 15
0x40	

10.1.4 Page Descriptor

The page descriptor specifies what physical page this specific descriptor maps to, as well as permissions and other flags

31..12	11..5	4	3	2..1	0
&adr	–	ex	rw	pl	p
Physical target page address[31:12]	Reserved	Execute	Writeable	Privilege level	Present

NOTE: `adr` **must** be page aligned (`0x1000`)

10.1.5 Setting up paging

The *target program*, after receiving control from the BLIP, should set up its own page directory structure according to its needs and point its `rpdp` register to the physical address of the level-0 page directory.

10.2 Virtual memory

The position (offset/index) of a page descriptor within the page directory as a whole determines the virtual address that gets mapped to the specific physical address that the page descriptor contains.

For example, the virtual address `0xABCDE000` would need a page descriptor at the 15th (`(0xABCDE000 >> 12) & 15 = 14`) position of the page table located at the 206th (`(0xABCDE000 >> 16) & 255 = 205`) position of a level-1 page directory, which would be addressed in the 172th (`0xABCDE000 >> 24 = 171`) position of the level-0 page directory.

This also translates to the last 8 bits (`0xFF000000`) of the address indicating the level-1 page directory index, the previous 8 bits (`0x00FF0000`) indicating the level-1 page directory index, and the previous 4 bits (`0x0000F000`) indicating the page table index, with the last 12 bits being the in-page address offset as pages are aligned to (`0x1000`).

10.3 Access protection

The nature of paging implicitly provides memory access protection features. Since, unless explicitly overriden or sharing physical pages between contexts, each context has its own virtual address space and virtual-physical address map, a malicious context cannot access memory of other contexts.

10.4 Level-0 physical memory map

This map is assigned by the hardware and can not be modified.

Base	Limit	Description	Type
0x0000 0000	0x0000 1000	–	Reserved
0x0000 1000	0xFFA0 0000	User RAM	User
0xFFA0 0000	0xFFA1 0000	BLIP ROM	Read-only, restricted
0xFFA1 0000	0xFFFD E000	–	Reserved
0xFFFD E000	0xFFFD E200	Level-1 memory map (10.5)	Hardware, restricted
0xFFFD E200	0xFFFE 0000	–	Reserved
0xFFFE 0000	0xFFFF FFFF	System RAM	System, restricted

NOTE: *System RAM* is a physically separate block of memory that is connected directly to the CPU as well as the MMU. It is used for direct register frame I/O and should be used by the *target program* for operations that require frequent and faster access than regular DRAM.

10.5 Level-1 physical memory map

This map routes addresses within *User*-designated spaces to optional interface host controllers, such as the SPPBC. It must be programmed by the

supervisor alongside interface-specific structures for the MMU to be able to discern *regular* memory addresses from addresses designated for use with Memory-Mapped Input/Output (MMIO).

By default, all addresses are mapped to *User RAM*. It must be able to map *at least* up to 64 address ranges total. All addresses must be `0x1000`-aligned.

Table 6: Level-1 memory map table

Offset	Value
0x00	Entry 0
0x08	Entry 1
0x10	Entry 2
...	...
0x1F8	Entry 63
0x200	

Table 7: Level-1 memory map table entry

63..44	43..24	23..16	15..8	7..0
&lm	&bs	–	idx	tg
Limit address[31:12]	Base address[31:12]	Reserved	Index	Target

Where *index* is the index of the target device to support addressing more than one device at a time and *target* must be one of the available hardware feature IDs (See 13.1) or 0 for none.

IMPORTANT: If the MMU tries to forwards an address to a *target* that does not exist in its feature set or an index that is out of range for its feature devices, it must raise a #IOP.

Overlapping address ranges are undefined behaviour and implementation-specific.

10.6 Memory Management Unit

The MMU is handles memory I/O operations and address routing.

- For *paging*, the MMU is responsible for translating virtual addresses to physical addresses
- For *I/O*, the MMU takes a requested physical address and acts like a *multiplexer* to correctly route the I/O operation in accordance with the physical memory layout, performing the requested operation

10.6.1 Operations

The MMU must provide the following operations for the CU:

Mnemonic	Description	Input	Output	INT on error
tavp	Translate Page Address Virtual to Physical	page-aligned virtual address, R/W/X	page-aligned physical address	#PLE
mread8	Read Physical Address 8	physical address	data (byte)	#MAE
mread16	Read Physical Address 16	physical address	data (half-word)	#MAE
mread32	Read Physical Address 32	physical address	data (word)	#MAE
mwrite8	Write Physical Address 8	physical address, data (byte)	–	#MAE
mwrite16	Write Physical Address 16	physical address, data (half-word)	–	#MAE
mwrite32	Write Physical Address 32	physical address, data (word)	–	#MAE

As stated in ¶5, the MMU allows only type-aligned addressing for I/O and **must** raise a **#MAE** on incorrect alignment.

A **#MAE** should also be raised if the CPU tries to access a restricted address range.

How the MMU communicates with other internal CPU components is up to the specific implementation.

11 Protection domains

There exist two privilege levels:

- Level 0, *user* level, is dedicated for regular user applications
- Level 1, *supervisor* level, is dedicated for the *target program* or OS kernel

along with three primary mechanisms contributing to system/cross-program security:

1. Some registers having restricted read-only access for level-0
2. Non-register I/O access to some memory regions being restricted to level-1
3. Per-page permissions

Enforcing privilege levels is the job of the CPU. This happens at five stages:

1. When writing to *rpl*, the CU checks if the requested privilege level is valid. If not, it raises a **#GPE**
2. When writing a register, the CU checks the current privilege level and, if it isn't level 1 and the target register is write-restricted, raises a **#GPE**
3. Upon receiving a `tavp` operation, the MMU reads the current privilege level as well as the requested page's privilege level, and raises a **#GPE** if they don't match
4. Upon receiving a `tavp` operation, the MMU checks the provided purpose (R/W/X), matches it with the requested page's permissions, and if they don't match it raises a **#GPE**
5. Upon receiving a `mread*` or `mwrite*` operation, the MMU checks if the provided physical address is in a restricted range. If it is, and the current privilege level is not level 1, it raises a **#GPE**

11.1 Protection level transfer

A transfer between protection level for the current *frame* can be performed at at any time by the *supervisor* by writing to the `rpl` register or for any frame by writing to its memory directly.

12 Context switching

A context switch can be performed at any time by the *supervisor* by writing to the `rfp` register.

13 Hardware features

13.1 Feature ID table

Bit	ID	Feature
0	0x01	PIT
1	0x02	UART
2	0x03	SPI
3	0x04	SPPB

13.2 Programmable Interval Timer

If a CPU lists the PIT as an available feature, it must have at least 4 total PITs. The actual number of available timers can be fetched through `cpuft` (See 13.2.6). I/O attempts with non-word-size data raise a **#IOP**.

- write `base + 0` - command

Table 8: Command frame

31..3	2..0
dt	cmd
Data	Command

A call to an unknown command raises **#UDO**.

13.2.1 0x00 Set mode

cmd	dt
0x00	Requested mode

Sets the target mode for this timer.

Table 9: Available timer modes

ID	Name	Description
0x00	Disabled	This timer is disabled (default)
0x01	One-shot	Calls a target INT on tick and disables itself
0x02	Tick	Calls a target INT on every tick

In any mode aside from *disabled*, the timer increments an internal counter by 1 for every passed tick. A mode change does not reset the counter.

Trying to set an invalid mode raises **#IOP**.

13.2.2 0x01 Set tick rate

cmd	dt
0x01	Target tick rate

13.2.3 0x02 Set target INT

cmd	dt
0x02	Target INT

Sets the target INT ID for this timer to call based on the current mode. A passed ID of 0 (default) disables any interrupt calling.

13.2.4 0x03 Get tick count

cmd	dt
0x03	Target register

Writes the tick count of this timer to the *target register*.

13.2.5 0x04 Reset counter

cmd	dt
0x04	–

Resets the internal counter of this timer to 0.

13.2.6 Feature information

31..8	7..0
–	num
Unused	Number of available timers

13.3 UART

UART uses PIT 0 as it's internal timer, with INT 0x90 as the callback interrupt.

- write `base + 0` (word cfg) - configure
- write `base + 4` (byte data) - TX
- read `base + 5` (byte data) - RX
- read `base + 6` (byte state) - get TX state
- read `base + 7` (byte state) - get RX state

NOTE: Reading is a non-blocking operation and outputs the byte currently stored in the controller, or 0 if none or the data has not yet been fully received. Another option to receive data from the UART controller is INT 0x91. (See below)

NOTE: Trying to transmit data (i.e. writing to TX) while existing data is still being transferred will result in a **#IOP** with rfi0 `0xFE`

13.3.1 TX state

0	Idle
1	Start (internal)
2	Sending data
3	Stop (internal)

13.3.2 RX state

0	Idle
1	Receiving data

13.3.3 Configuration packet

31..26	25..24	23..0
–	pr	bd
Reserved	Parity bit	Baud rate

Where *parity bit* can be one of:

0	None
1	Odd
2	Even

And *baud rate* depends on the implementation. Every implementation must support at least a baud rate of 9600.

NOTE: An invalid *parity bit* is corrected to *None*, and an unsupported *baud rate* is corrected to 9600.

13.3.4 Interrupts

INT	Description	rfi0	rfi1
0x90	Timer tick	–	–
0x91	Data receive	byte data	–

13.4 SPI

The built-in SPI controller always acts as a *master* and must support at least 2 *slave* devices. Operations on the controller and communication between

the controller and devices is done through I/O on the mapped memory address, here referred to as the *base address* (See 10.4):

- write `base + 0` (byte id) - select target device
- ...

14 Structures

14.1 CPUID

